

리눅스 4.1 - 인터럽트 및 용어정리

남궁명수

NVMe(Non-Volatile Memory Express)

- SSD 자체나 연결선이 아니라, 운영체제가 PCIe 기반의 고속 저장장치와 통신하는 규격(프로토콜)이다.
- PCIe가 데이터를 운반하는 도로라면, NVMe는 그 도로에서 사용하는 통신 규칙이다.

프로그램

↓ read(), write()

리눅스 파일시스템

↓

NVMe 장치 드라이버

↓ NVMe 명령

PCIe

↓ 전송

NVMe SSD

M.2 → 장치의 크기와 커넥터 규격

PCIe → 데이터가 이동하는 통로

NVMe → SSD를 제어하는 통신 규칙

SSD → 데이터를 저장하는 실제 장치

[기존 SATA SSD와 무엇이 다를까?]

기존 SATA SSD는 보통 AHCI라는 통신 규격을 사용한다.

[SATA SSD]

운영체제

↓ AHCI 명령

SATA 컨트롤러

↓ SATA 케이블

SSD

AHCI는 원래 기계식 HDD를 대상으로 설계됐다.

HDD는 디스크 헤드가 움직이기 때문에 한꺼번에 엄청나게 많은 요청을 처리할 필요가 크지 않았다.

반면, NVMe는 처음부터 빠른 플래시 메모리와 멀티코어 CPU를 고려해서 설계됐다.

[NVMe SSD]

운영체제

↓ NVMe 명령

PCIe 컨트롤러

↓ PCIe 레인

SSD

구분	SATA + AHCI	PCIe + NVMe
전송 통로	SATA	PCIe
프로토콜	AHCI	NVMe
설계 대상	HDD 중심	플래시 SSD 중심
명령 큐	1개	여러 개
큐당 명령 수	최대 32개	최대 65,536개 수준
병렬 처리	제한적	매우 유리
지연 시간	상대적으로 큼	상대적으로 작음

[NVMe는 어떻게 요청을 처리할까?]

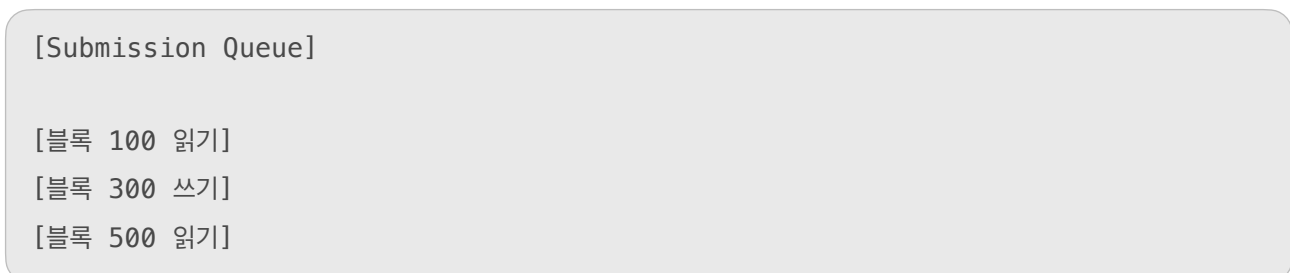
NVMe의 핵심은 Submission Queue와 Completion Queue다.



[Submission Queue]

운영체제가 SSD에 처리할 명령을 넣는 공간이다.

예를 들면:



[Completion Queue]

SSD가 처리를 마친 명령의 결과를 넣는 공간이다.

[Completion Queue]

[블록 100 읽기 완료]

[블록 300 쓰기 완료]

이 큐들은 보통 RAM에 생성되고, 운영체제와 SSD가 함께 사용한다.

[실제 읽기 과정]

프로그램이 파일을 읽는 과정을 단순화하면 다음과 같다.

[1] 프로그램이 read() 호출

```
read(fd, buffer, 4096);
```

[2] 커널이 NVMe 명령 생성

커널의 NVMe 드라이버가 다음과 같은 정보를 준비한다.

명령: Read

저장장치 위치: 특정 논리 블록

크기: 4KiB

RAM 위치: 데이터를 저장할 메모리 주소

[3] Submission Queue에 등록

커널이 요청을 RAM의 Submission Queue에 넣은 뒤, SSD에 새 요청이 있다는 걸을 알린다.

이 알림에 사용되는 레지스터를 보통 Doorbell Register라고 한다.

커널: “새 명령을 큐에 넣었어.”

SSD: “큐를 확인할게.”

[4] SSD가 DMA로 데이터 전송

SSD는 읽은 데이터를 CPU가 하나씩 복사해 주기를 기다리지 않는다.

DMA를 사용해 RAM으로 직접 전송할 수 있다.

NVMe SSD — DMA → RAM

여기서도 실제 데이터는 PCIe를 통해 이동한다.

[5] 완료 알림

SSD가 Completion Queue에 결과를 기록하고, 보통 MSI-X 인터럽트를 이용해 CPU에 작업 완료를 알린다.

SSD

|

| MSI-X 인터럽트

▼

CPU: “읽기가 끝났구나.”

그 후 커널이 기다리던 프로세스를 깨우고 read()가 반환된다.

[왜 큐가 여러 개 필요할까?]

CPU 코어가 여러 개라면 코어마다 독립적인 큐를 사용할 수 있다.

```
CPU Core 0 — Queue 0 —┐
CPU Core 1 — Queue 1 —┐
CPU Core 2 — Queue 2 —┴─ NVMe SSD
CPU Core 3 — Queue 3 —┐
```

하나의 큐만 공유하면 여러 코어가 큐를 사용하기 위해 락을 두고 경쟁할 수 있다.

코어별로 큐를 사용하면

- 여러 I/O 요청을 동시에 제출할 수 있음
- 락 경쟁이 줄어듦
- 멀티코어를 효율적으로 활용할 수 있음
- 많은 I/O 요청을 병렬로 처리할 수 있음

[NVMe SSD 내부]

NVMe SSD도 단순한 메모리 칩은 아니다.

PCIe

|

NVMe 컨트롤러

├ 명령 큐 관리

├ 논리 주소를 실제 플래시 위치로 변환

├ 오류 정정

├ 캐시 관리

└ 여러 NAND 채널 병렬 처리

|

├ NAND Flash

├ NAND Flash

├ NAND Flash

└ NAND Flash

운영체제는 SSD의 물리적인 NAND 위치를 직접 지정하지 않고, LBA(Logical Block Address)라는 논리 블록 번호를 사용한다. SSD 컨트롤러가 LBA를 실제 NAND 위치로 변환하는데, 이를 담당하는 계층이 FTL(Flash Translation Layer)이다.

[M.2 SSD는 모두 NVMe일까?]

M.2는 외형과 커넥터 규격이므로 M.2 SSD라도 SATA 방식일 수 있다.

M.2 SATA SSD → SATA 통로 + AHCI

M.2 NVMe SSD → PCIe 통로 + NVMe

겉모습이 비슷해도 내부 통신 방식과 성능이 다를 수 있으므로 제품 사양에서 SATA인지 PCIe NVMe인지 확인해야 한다.

NAND(NAND Flash Memory)

- ‘NAND’ 자체는 논리회로인 Not-And(부정 앤드) 게이트 구조에서 유래한 이름이다. 전원 공급이 끊겨도 데이터가 보존되는 비휘발성 메모리 장치이다.
- 전원이 꺼져도 데이터가 지워지지 않는 비휘발성 NAND 플래시 메모리 하드웨어를 제어하고 관리하는 시스템 및 드라이버 영역을 뜻한다. 주로 임베디드 기기나 SSD에서 파일 시스템을 올릴 때 사용한다.

인터럽트 컨트롤러(Interrupt Controller)

- 컴퓨터 시스템 안팎의 여러 장치(키보드, 마우스 타이머, 통신 장치 등)가 보내는 비동기적 끼어들기 요청(인터럽트)을 모아서 정리하고, 우선순위를 매겨 중앙처리장치(CPU)에 순서대로 전달하는 하드웨어 제어장치

인터럽트 핸들러(Interrupt Handler)

- CPU가 하던 일을 멈추고 긴급한 신호(인터럽트)를 처리하기 위해 실행하는 특별한 프로그램 코드나 함수이다.
- 인터럽트 서비스 루틴(ISR)이라고도 부르며, 키보드 입력이나 장치 오류 같은 상황을 해결한다.

MSI와 MSI-X

MSI와 MSI-X는 PCIe 장치가 CPU에 “작업이 완료됐다”라고 알리는 인터럽트 전달 방식이다.

MSI는 Message Signaled Interrupt,
MSI-X는 MSI를 확장한 방식이다.

예를 들어, NVMe SSD가 읽기 작업을 완료하면 CPU에 알려야 한다.

CPU: “데이터를 읽어 와.”

↓

NVMe SSD

↓ 작업 완료

CPU: “완료됐구나!” ← 인터럽트

이때 완료 사실을 전달하는 방법이 INTx, MSI, MSI-X

[1] [기존 방식: INTx]

예전 PCI 장치는 인터럽트 전용 신호선을 사용했다.

PCI 장치 — 인터럽트 신호선 → 인터럽트 컨트롤러 → CPU

장치가 신호선의 전압을 바꾸면 인터럽트가 발생하는 방식이다.

하지만 물리적인 인터럽트 선은 한정되어 있기 때문에 여러 장치가 같은 인터럽트 선을 공유할 수 있었다.

랜카드 ┌
사운드카드 ┤ 공유 인터럽트 선 → CPU
저장장치 └

CPU는 인터럽트가 발생해도 어떤 장치가 요청했는지 즉시 알기 어렵다.

CPU: “누가 인터럽트를 발생시켰지?”

랜카드 확인

사운드카드 확인

저장장치 확인

이를 공유 인터럽트라고 한다.

장치를 하나씩 확인해야 해서 불필요한 작업이 발생할 수 있다.

[2] [MSI: 메시지로 인터럽트 전달]

MSI(Message Signaled Interrupt)는 별도의 인터럽트 신호선을 사용하지 않는다.

PCIe 장치가 특정 메모리 주소에 특별한 값을 쓰면, 인터럽트 컨트롤러가 이를 CPU 인터럽트로 해석한다.

PCIe 장치
|
| 특정 주소에 특정 값 쓰기
▼
인터럽트 컨트롤러
|
| 인터럽트 전달
▼
CPU

여기서 말하는 메시지는 일반적인 문자열이 아니라 다음과 같은 작은 쓰기 요청이다.

주소: 어느 CPU 또는 인터럽트 컨트롤러로 보낼 것인가

데이터: 어떤 인터럽트 벡터를 발생시킬 것인가

PCIe 장치는 이 정보를 PCIe 메모리 쓰기 패킷으로 보낸다.

장치

|

| PCIe Memory Write 패킷

|

주소 + 데이터



CPU의 인터럽트 처리 구조

즉, MSI의 핵심은 다음과 같다.

물리적인 인터럽트 선을 흔드는 대신 PCIe 패킷을 보내 인터럽트를 발생시킨다.

MSI의 장점

- 물리적인 인터럽트 선이 필요하지 않음
- 인터럽트 공유 문제가 줄어듦
- 장치가 여러 인터럽트 벡터를 사용할 수 있음
- 특정 CPU 코어로 인터럽트를 전달할 수 있음

[3] [인터럽트 벡터란]

인터럽트 벡터는 CPU가 어떤 인터럽트 처리 코드를 실행할지 구분하는 번호이다.

인터럽트 벡터 45 → NVMe 완료 처리 함수

인터럽트 벡터 46 → 랜카드 수신 처리 함수

인터럽트 벡터 47 → USB 처리 함수

실제로는 운영체제가 장치를 초기화하면서 인터럽트 벡터와 처리 함수인 핸들러를 연결해 둔다.

인터럽트 벡터



운영체제의 인터럽트 핸들러



해당 장치의 완료 처리

MSI의 메시지에 들어 있는 데이터가 어떤 인터럽트 벡터를 발생시킬지 나타내는 데 사용된다.

[4] [MSI-X: MSI의 확장 방식]

MSI-X는 MSI보다 훨씬 많은 인터럽트 벡터를 유연하게 사용할 수 있도록 확장한 방식이다.

구분	MSI	MSI-X
최대 인터럽트 벡터	최대 32개	최대 2,048개
벡터 개수 조건	일반적으로 2의 거듭제곱 단위	필요한 만큼 유연하게 할당
벡터별 주소·데이터	제한적	개별 설정 가능
개별 인터럽트 마스크	기능이 제한적	벡터별로 가능
주 사용 대상	일반 PCIe 장치	NVMe, 고속 랜카드 등

여기서 마스킹은 특정 인터럽트를 일시적으로 막는다는 뜻이다.

Vector 40 → 허용

Vector 41 → 일시적으로 차단

Vector 42 → 허용

MSI-X는 벡터마다 독립적으로 설정하고 차단할 수 있다.

[5] [MSI-X Table]

MSI-X를 지원하는 장치는 MSI-X Table이라는 표를 가지고 있다.

보통 장치의 BAR를 통해 접근 가능한 장치 메모리 공간에 존재한다.

각 항목에는 대략 다음 정보가 들어간다.

MSI-X Table

항목	메시지 주소	메시지 데이터	마스크
0	CPU 대상 정보	Vector 40	허용
1	CPU 대상 정보	Vector 41	허용
2	CPU 대상 정보	Vector 42	차단

리눅스 장치 드라이버가 장치를 초기화하면서 이 테이블을 설정한다.

장치는 작업이 끝났을 때 해당 항목의 주소와 데이터를 사용해 PCIe 쓰기 요청을 보낸다.

[6] [NVMe에서 MSI-X를 사용하는 이유]

NVMe는 여러 개의 I/O 큐를 사용할 수 있다.

CPU Core 0 — NVMe Queue 0

CPU Core 1 — NVMe Queue 1

CPU Core 2 — NVMe Queue 2

CPU Core 3 — NVMe Queue 3

모든 큐가 인터럽트 하나를 공유하면 다음과 같은 문제가 생긴다.

모든 완료 인터럽트

↓

CPU Core 0

Core 0에 인터럽트가 몰릴 수 있고, 어떤 큐에서 작업이 끝났는지 추가로 확인해야 한다.

MSI-X를 사용하면 큐마다 서로다른 벡터와 CPU 코어를 연결할 수 있다.

```
NVMe Queue 0 완료 — Vector 40 → CPU Core 0
NVMe Queue 1 완료 — Vector 41 → CPU Core 1
NVMe Queue 2 완료 — Vector 42 → CPU Core 2
NVMe Queue 3 완료 — Vector 43 → CPU Core 3
```

이렇게 하면

- 여러 코어가 완료 처리를 분산해서 수행
- 한 코어에 인터럽트가 집중되는 현상 감소
- 큐를 찾기 위한 추가 확인 감소
- CPU 캐시 지역성 향상
- 높은 병렬 처리 성능 확보

[7] NVMe 읽기부터 MSI-X까지의 전체 흐름

1. 프로그램이 read() 호출
- ↓
2. 커널이 NVMe 명령 생성
- ↓
3. Submission Queue에 명령 등록
- ↓
4. 커널이 Doorbell 레지스터 기록
- ↓
5. NVMe SSD가 명령 처리
- ↓
6. DMA를 통해 데이터를 RAM에 기록
- ↓
7. Completion Queue에 완료 정보 기록
- ↓
8. SSD가 MSI-X 메시지 전송
- ↓
9. 지정된 CPU 코어에서 인터럽트 발생
- ↓
10. NVMe 드라이버가 Completion Queue 확인

중요한 점은 MSI-X 메시지에 실제 파일 데이터가 들어가는게 아니라는 것이다.

```
DMA    → 실제 데이터를 SSD에서 RAM으로 전달
MSI-X  → 데이터 처리가 끝났다고 CPU에 알림
```

[8] 리눅스에서 확인하기

PCI 장치가 어떤 인터럽트 방식을 사용하는지 다음 명령으로 확인할 수 있다.

```
sudo lspci -vv
```

출력에서 다음과 같은 항목을 찾으면 된다.

```
Capabilities: [b0] MSI-X: Enable+ Count=65 Masked-
```

- MSI-X: MSI-X 지원
- Enable+: 현재 활성화됨
- Count=65: MSI-X 테이블 엔트리 개수
- Masked-: 전체 MSI-X가 차단된 상태는 아님

현재 인터럽트가 CPU 코어별로 얼마나 처리됐는지는 다음으로 확인할 수 있다.

```
cat /proc/interrupts
```

NVMe 장치라면 다음처럼 여러 큐가 보일 수 있다.

	CPU0	CPU1	CPU2	CPU3
nvme0q0	120	0	0	0
nvme0q1	0	3500	0	0
nvme0q2	0	0	3200	0
nvme0q3	0	0	0	3400

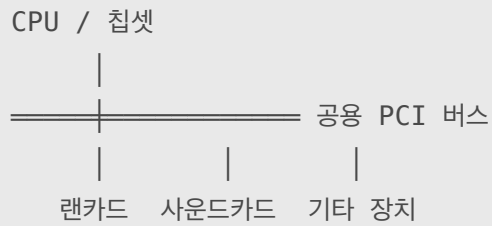
nvme0q1, nvme0q2 등은 NVMe 큐별 인터럽트를 나타내며, 여러 CPU 코어에 분산된 모습을 확인할 수 있다.

BAR(Base Address Register, 기본 주소 레지스터)

- 운영체제나 바이오스가 각 하드웨어 장치(그래픽 카드, NVMe SSD 등)의 내부 메모리 및 I/O 공간에 접근할 수 있도록 시스템 주소 공간을 할당하고 매핑해주는 핵심 구성 요소이다.

PCI(Peripheral Component Interconnect)

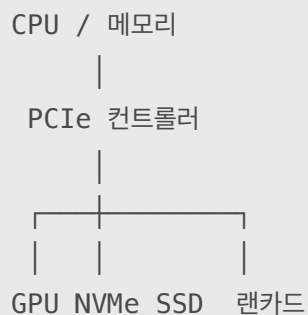
- 컴퓨터 메인보드에서 CPU와 그래픽카드, 랜카드 같은 하드웨어 주변장치를 서로 연결하는 표준 규격과, 리눅스 커널이 이 장치들을 관리하는 하드웨어 서브 시스템을 뜻한다.
- 하나의 도로를 여러 장치가 공유



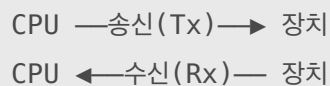
- PCI는 여러 개의 신호 선을 사용해 여러 비트를 한 번에 보내는 병렬 통신 방식이다.
 - 여러 장치가 하나의 버스를 공유
 - 한 장치가 통신하는 동안 다른 장치는 기다릴 수 있음
 - 장치가 많아지면 대역폭 경쟁 발생
 - 병렬 신호선 간 타이밍을 정확히 맞추기 어려움
 - 현재는 대부분 PCIe로 대체

PCIe(Peripheral Component Interconnect Express, 주변장치 상호연결 익스프레스)

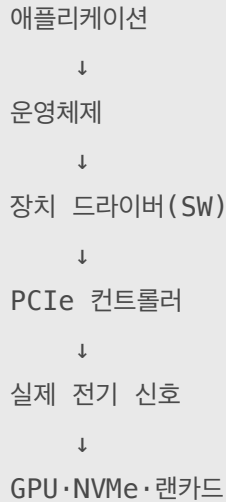
- 소프트웨어가 아니라, CPU와 장치를 연결하는 고속 하드웨어 통신 규격이다.(데이터가 오가는 도로(버스))
- 장치마다 전용 도로 사용



- PCIe는 실제로 신호가 지나가는 전기적 배선(레인, Lane)이 존재한다.
 - PCIe x1: 레인 1개
 - PCIe x4: 레인 4개 - NVMe SSD에서 혼함
 - PCIe x8: 레인 8
 - PCIe x16: 레인 16개 - GPU에서 혼함
- 레인 하나는 다음 처럼 송신용과 수신용 신호선으로 구성돼서 동시에 양방향 통신이 가능하다.



- 다만 PCIe가 동작하려면 소프트웨어도 함께 필요하다.



- PCIe 자체는 장치를 연결하는 하드웨어 인터페이스이자 통신 규격이고, 실제 선인 레인을 통해 데이터를 전달한다.
- 장치 드라이버는 PCIe 장치를 제어하는 소프트웨어다.

[병렬인 PCI보다 직렬인 PCIe가 더 빠른 이유]

겉으로 보면 여러 비트를 동시에 보내는 PCI가 더 빠를 것 같지만, 병렬 통신에는 문제가 있다.

PCI 병렬 신호



신호선이 많아질수록 각 신호의 도착 시간을 맞추기 어려워져서 전송 속도를 무작정 높일 수 없다.

PCIe는 적은 신호선을 이용해 데이터를 매우 빠르게 연속 전송한다.

PCIe 직렬 신호

1 → 0 → 1 → 1 → 0 → 1

필요하면 이런 통로인 레인 자체를 여러 개 사용하면 된다.

PCIe x4

